

184.686 VU Database Systems

Anfrageoptimierung

Katja Hose

Institute für Logic and Computation

Sommersemester 2026



Informatics

- 1 Motivation und Einführung
 - Anfragebearbeitung
 - Anfrageoptimierung
- 2 Logische (heuristische) Anfrageoptimierung
 - Äquivalenzerhaltende Transformationsregeln
 - Phasen der logischen Anfrageoptimierung
- 3 Implementierung von Operatoren
 - Selektion (Access Paths)
 - Join-Algorithmen

Ausführen einer SQL-Anfrage

Die Klauseln werden in folgender Reihenfolge angegeben.

- SELECT *column(s)*
- FROM *table list*
- WHERE *condition*
- GROUP BY *grouping column(s)*
- HAVING *group condition*
- ORDER BY *sort list*

SQL ist deklarativ!

Schritte der Anfragebearbeitung

Anfrage in einer abstrakten Anfragesprache

↓

Scanning, Parsing, and Semantic Analysis

↓

Intermediate Query Plan

Query Optimizer

↓

Execution Plan

Code Generator

↓

Code for Query Execution

Runtime Database Processor

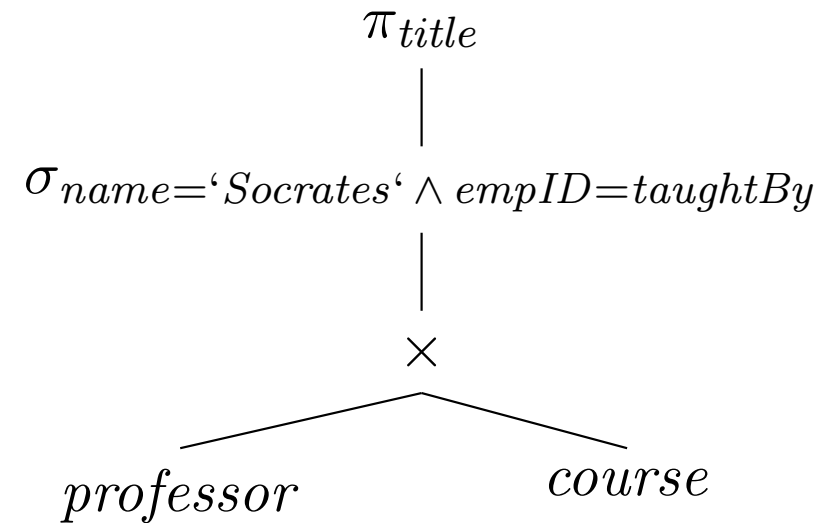
↓

Anfrageergebnis

Parsen einer Anfrage in einen initialen Anfrageplan

SELECT title
FROM professor, course
WHERE name='Socrates' AND
 empID = taughtBy;

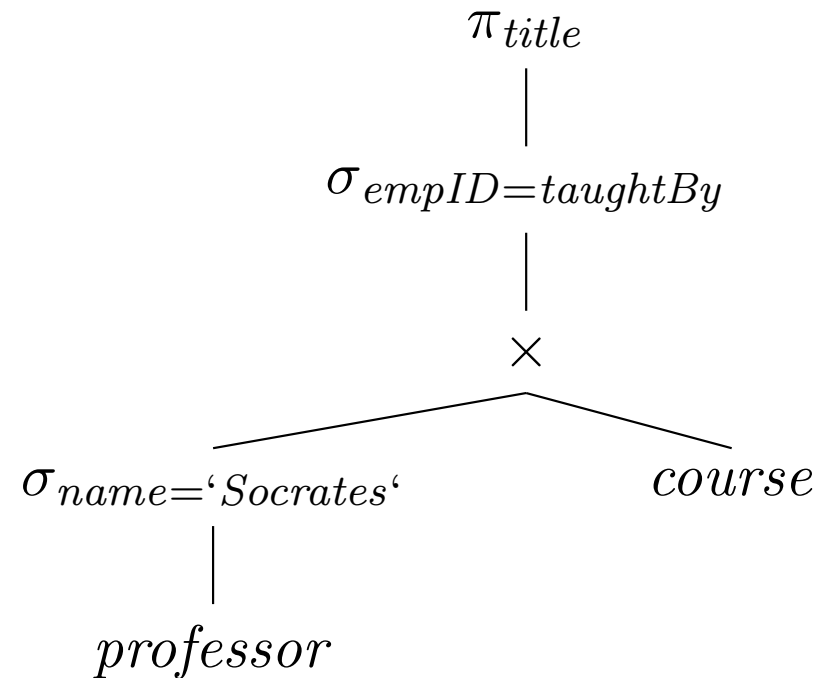
⇒



$\pi_{title}(\sigma_{name='Socrates' \wedge empID=taughtBy}(professor \times course))$

Alternativer Anfrageplan

SELECT title
FROM professor, course
WHERE name='Socrates' AND
 empID = taughtBy;

 $\Rightarrow$ 
$$\pi_{title}(\sigma_{empID=taughtBy}(\sigma_{name='Socrates'} professor \times course))$$

Anfrageoptimierung

Alternativen

- Äquivalente Ausführungspläne
- Algorithmen zur Ausführung von Algebraoperatoren
- Methoden, um auf Relationen zuzugreifen (Indexe)

Bei gleichem Ergebnis können Ausführungskosten sehr unterschiedlich sein.

Theorie vs. Realität

Es ist nicht die Aufgabe des Benutzers, „effiziente“ Anfragen zu schreiben, sondern die Aufgabe der Anfrageoptimierung, effiziente Ausführungspläne zu finden!

Aber in der Realität... Optimierer sind nicht perfekt.

Anfrageausführungskosten

Kostenmodell

- Gesamte Zeit bis das Anfrageergebnis vorliegt (**Response Time**)
- Viele Faktoren tragen dazu bei
 - Festplattenzugriff
 - CPU-Kosten
 - Netzwerkkommunikation
 - Aktueller Query-Load
 - Parallelisierung
- Festplattenzugriff dominiert
 - Block Access Time: Seek Time, Rotation Time

- 2 Logische (heuristische) Anfrageoptimierung
 - Äquivalenzerhaltende Transformationsregeln
 - Phasen der logischen Anfrageoptimierung

Logische Anfrageoptimierung

Logische Anfrageoptimierung

- Grundlage: Äquivalenzerhaltende Transformationsregeln
- Algebraische Transformationen bilden den Suchraum
- Gegeben sei ein initialer algebraischer Ausdruck:
verwende äquivalenzerhaltende Transformationsregeln, um neue Ausdrücke abzuleiten

Was ist ein guter Plan?

- Genaue Entscheidung ohne Kostenfunktion nicht möglich

⇒ logische Anfrageoptimierung basiert auf Heuristiken

Hauptziel der logischen Anfrageoptimierung

Größe von Zwischenergebnissen reduzieren

Äquivalenzerhaltende Transformationsregeln

Aufbrechen von Konjunktionen in Selektionsprädikaten

$$\sigma_{c_1 \wedge c_2 \wedge \dots \wedge c_n}(R) \equiv \sigma_{c_1}(\sigma_{c_2}(\dots(\sigma_{c_n}(R))\dots))$$

σ ist kommutativ

$$\sigma_{c_1}(\sigma_{c_2}(R)) \equiv \sigma_{c_2}(\sigma_{c_1}(R))$$

π -Kaskaden

If $L_1 \subseteq L_2 \subseteq \dots \subseteq L_n$ dann gilt

$$\pi_{L_1}(\pi_{L_2}(\dots(\pi_{L_n}(R))\dots)) \equiv \pi_{L_1}(R)$$

Äquivalenzerhaltende Transformationsregeln

Vertauschen der Reihenfolge von σ und π

Falls die Selektion sich nur auf Attribute $A_1, \dots, A_n$ der Projektionsliste bezieht, können die beiden Operationen vertauscht werden:

$$\pi_{A_1, \dots, A_n}(\sigma_c(R)) \equiv \sigma_c(\pi_{A_1, \dots, A_n}(R))$$

$\cup, \cap$ und $\bowtie$ sind kommutativ

$$R \bowtie_c S \equiv S \bowtie_c R$$

Äquivalenzerhaltende Transformationsregeln

Vertauschen von σ und $\bowtie$

Falls das Selektionsprädikat c nur auf Attribute der Relation R zugreift, kann man die beiden Operationen vertauschen:

$$\sigma_c(R \bowtie_j S) \equiv \sigma_c(R) \bowtie_j S$$

Falls das Selektionsprädikat c eine Konjunktion der Form $c_1 \wedge c_2$ ist und c_1 sich nur auf Attribute aus R und c_2 sich nur auf Attribute aus S bezieht, gilt folgende Äquivalenz:

$$\sigma_c(R \bowtie_j S) \equiv \sigma_{c_1}(R) \bowtie_j \sigma_{c_2}(S)$$

Äquivalenzerhaltende Transformationsregeln

Vertauschen von π und $\bowtie$

Gegeben sei Projektionsliste $L = \{A_1, \dots, A_n, B_1, \dots, B_m\}$, wobei A_i Attribute aus R und B_i Attribute aus S sind. Falls sich das Joinprädikat c nur auf Attribute aus L bezieht, gilt folgende Umformung:

$$\pi_L(R \bowtie_c S) \equiv (\pi_{A_1, \dots, A_n}(R)) \bowtie_c (\pi_{B_1, \dots, B_m}(S))$$

Äquivalenzerhaltende Transformationsregeln

Äquivalenzerhaltende Transformationsregeln

$\bowtie, \cap, \cup$ sind (jeweils einzeln betrachtet) assoziativ

Wenn also Φ eine dieser Operationen bezeichnet, so gilt:

$$(R \Phi S) \Phi T \equiv R \Phi (S \Phi T)$$

σ ist distributiv mit $\cap, \cup, -$

Wenn also Φ eine dieser Operationen bezeichnet, so gilt:

$$\sigma_c(R \Phi S) \equiv (\sigma_c(R)) \Phi (\sigma_c(S))$$

π ist distributiv mit $\cup$

$$\pi_c(R \cup S) \equiv (\pi_c(R)) \cup (\pi_c(S))$$

- 2 Logische (heuristische) Anfrageoptimierung
 - Äquivalenzerhaltende Transformationsregeln
 - Phasen der logischen Anfrageoptimierung

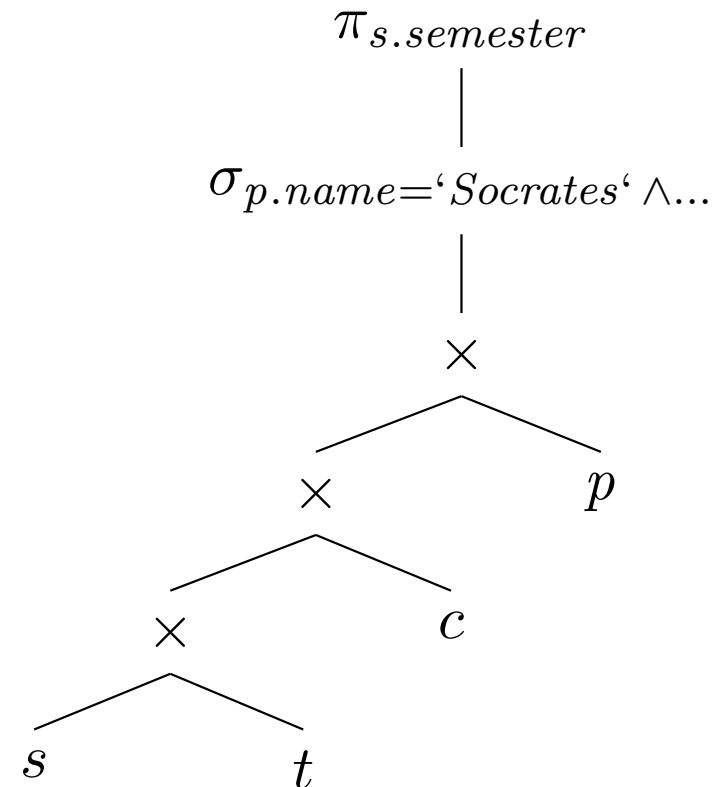
Phasen der logischen Anfrageoptimierung

- ① Aufbrechen von Selektionen
- ② Verschieben der Selektionen so weit wie möglich nach unten (pushing selections)
- ③ Joins einführen (Zusammenfassen von Selektionen und Kreuzprodukten)
- ④ Join-Reihenfolge bestimmen, so dass möglichst kleine Zwischenergebnisse entstehen
Heuristik: Joins mit Input von Selektionen vor anderen Joins auswerten
- ⑤ ggf. Einführen von Projektionen
- ⑥ Verschieben der Projektionen so weit wie möglich nach unten
Nicht immer nötig

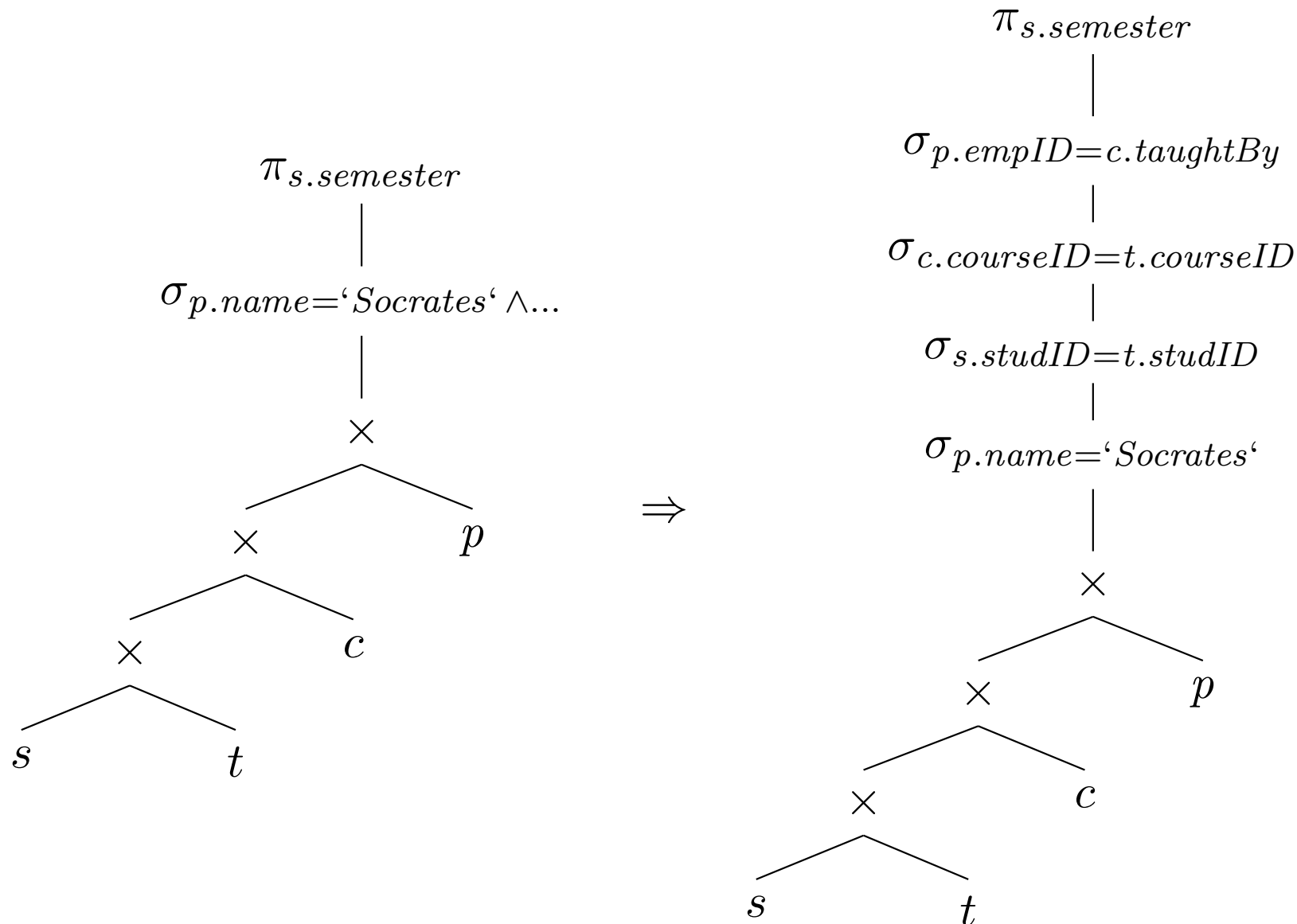
Beispiel

SELECT DISTINCT s.semester
FROM student s, takes t,
course c, professor p
WHERE p.name='Socrates' AND
c.taughtBy = p.empID AND
c.courseID = t.courseID AND
t.studID = s.studID;

⇒

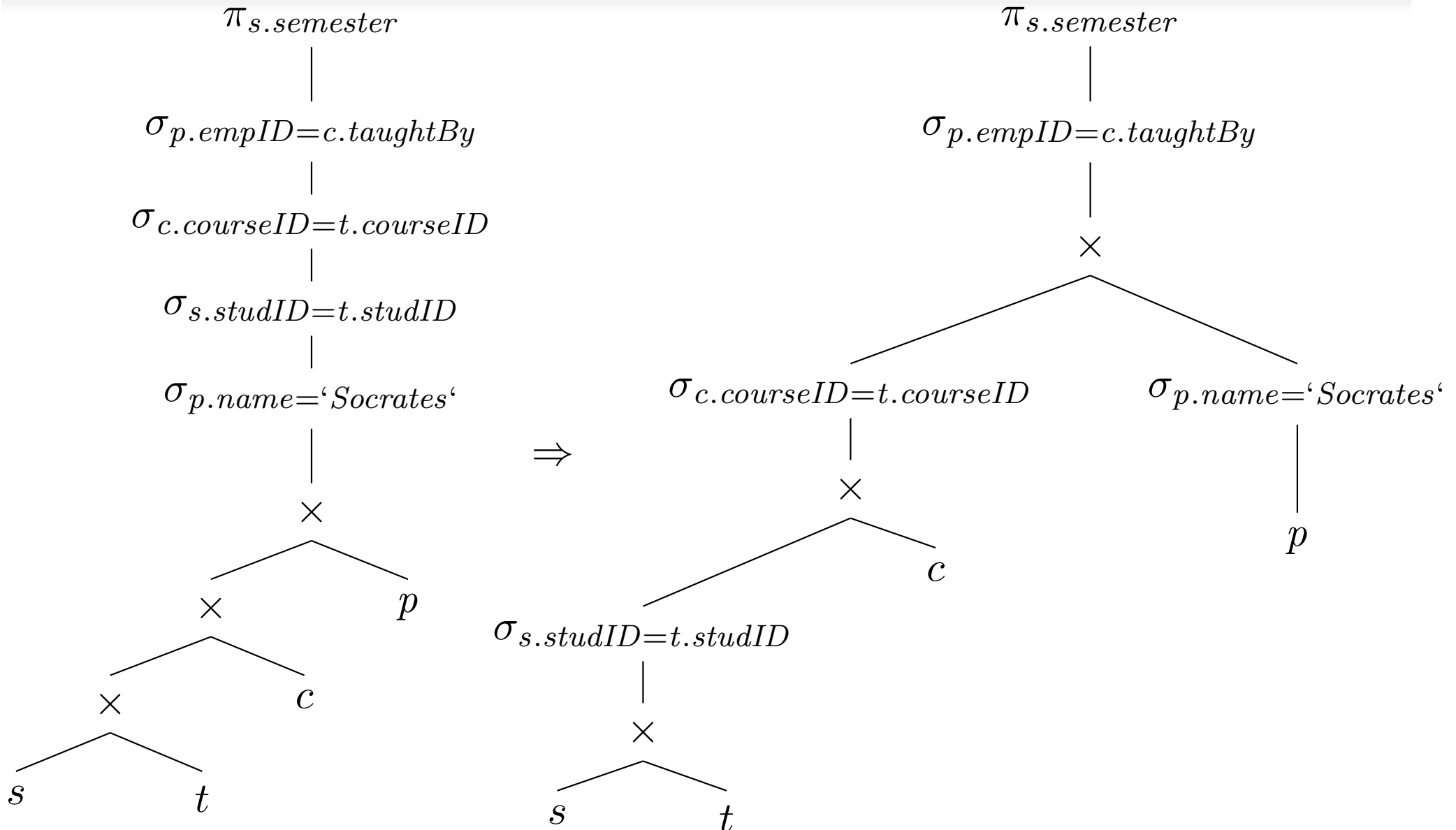


Aufbrechen von Selektionen



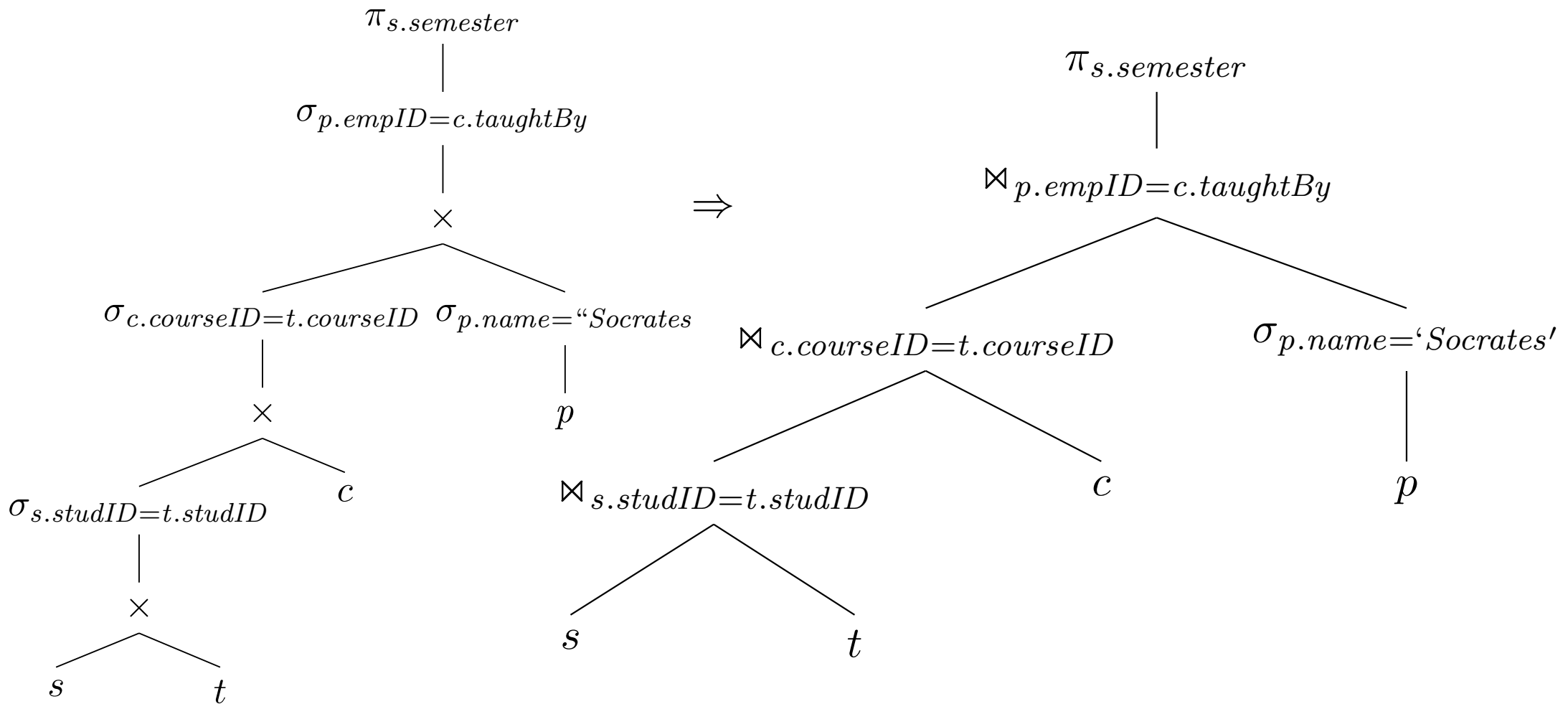
Phasen der logischen Anfrageoptimierung

Verschieben von Selektionen

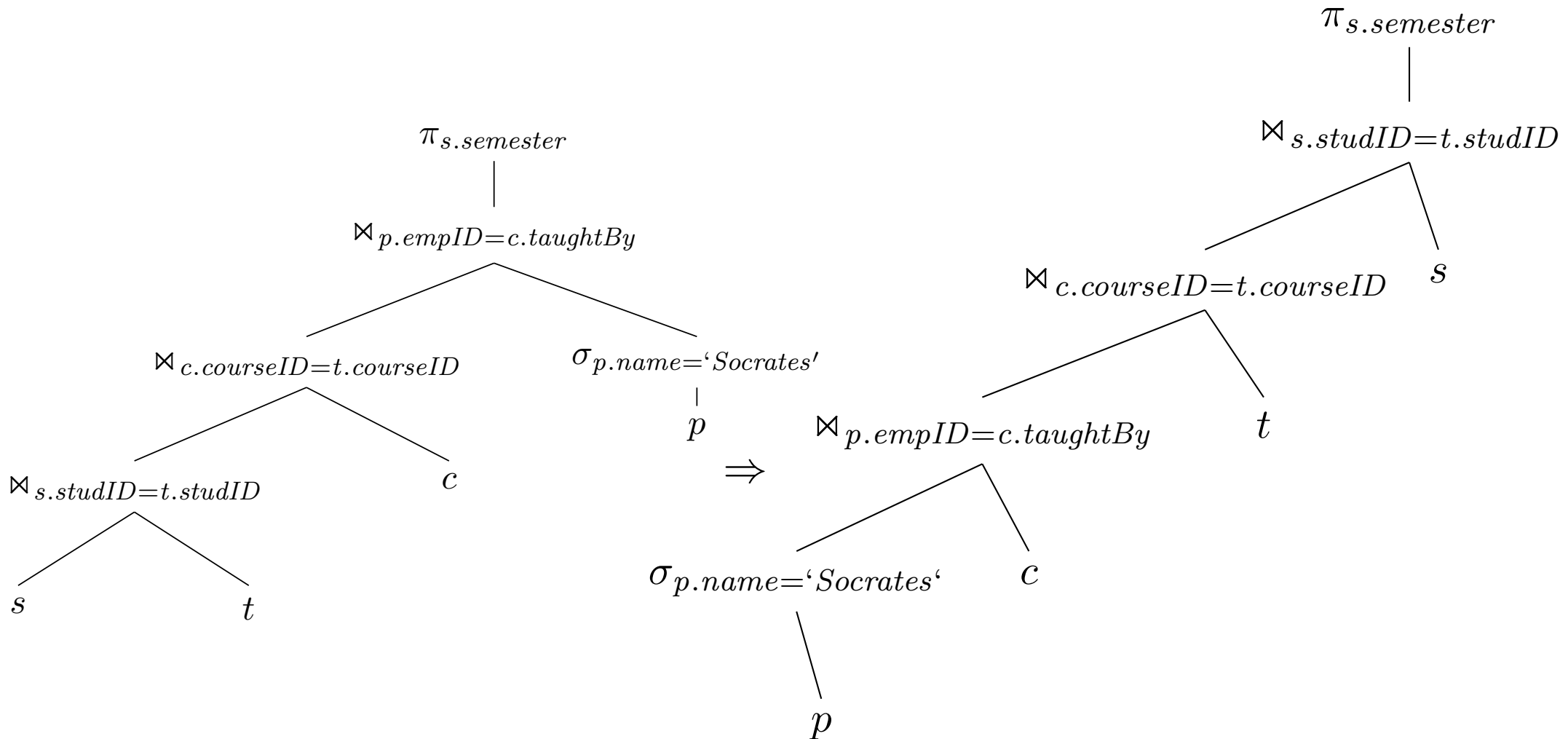


Phasen der logischen Anfrageoptimierung

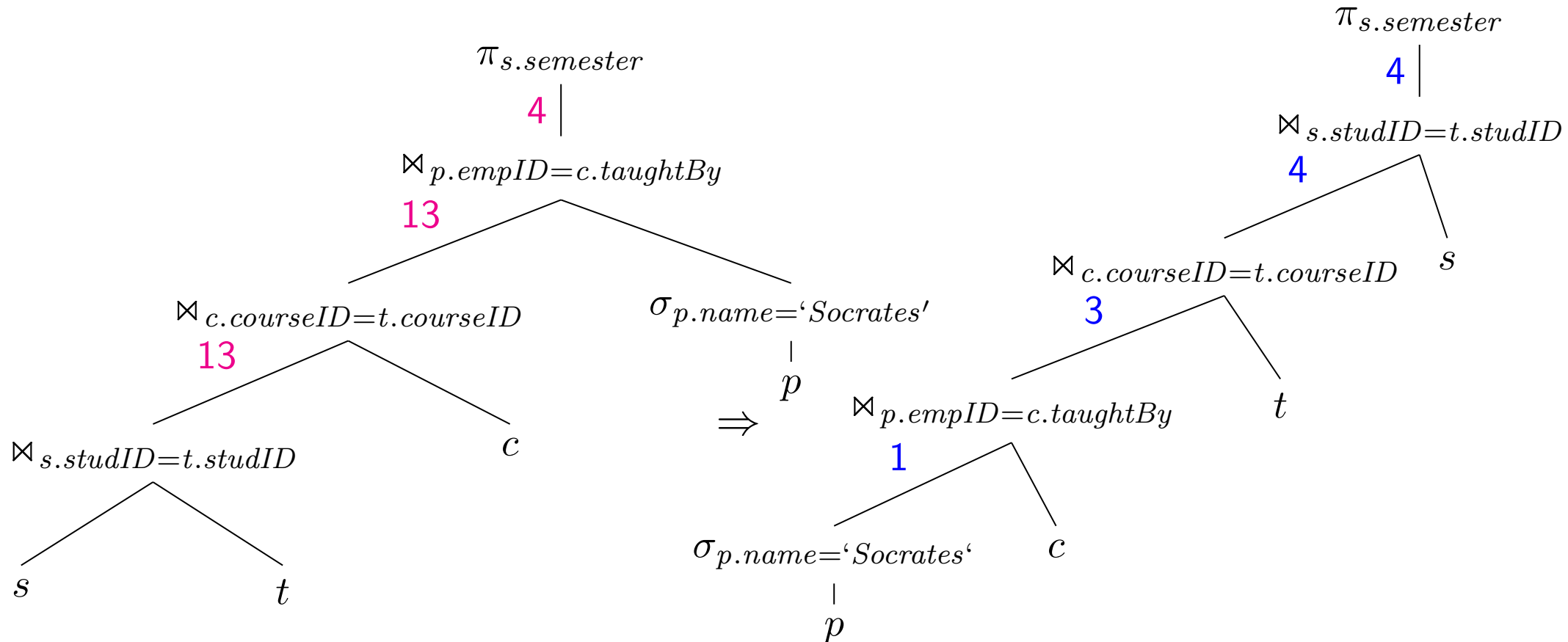
Joins einführen



Joinreihenfolge bestimmen

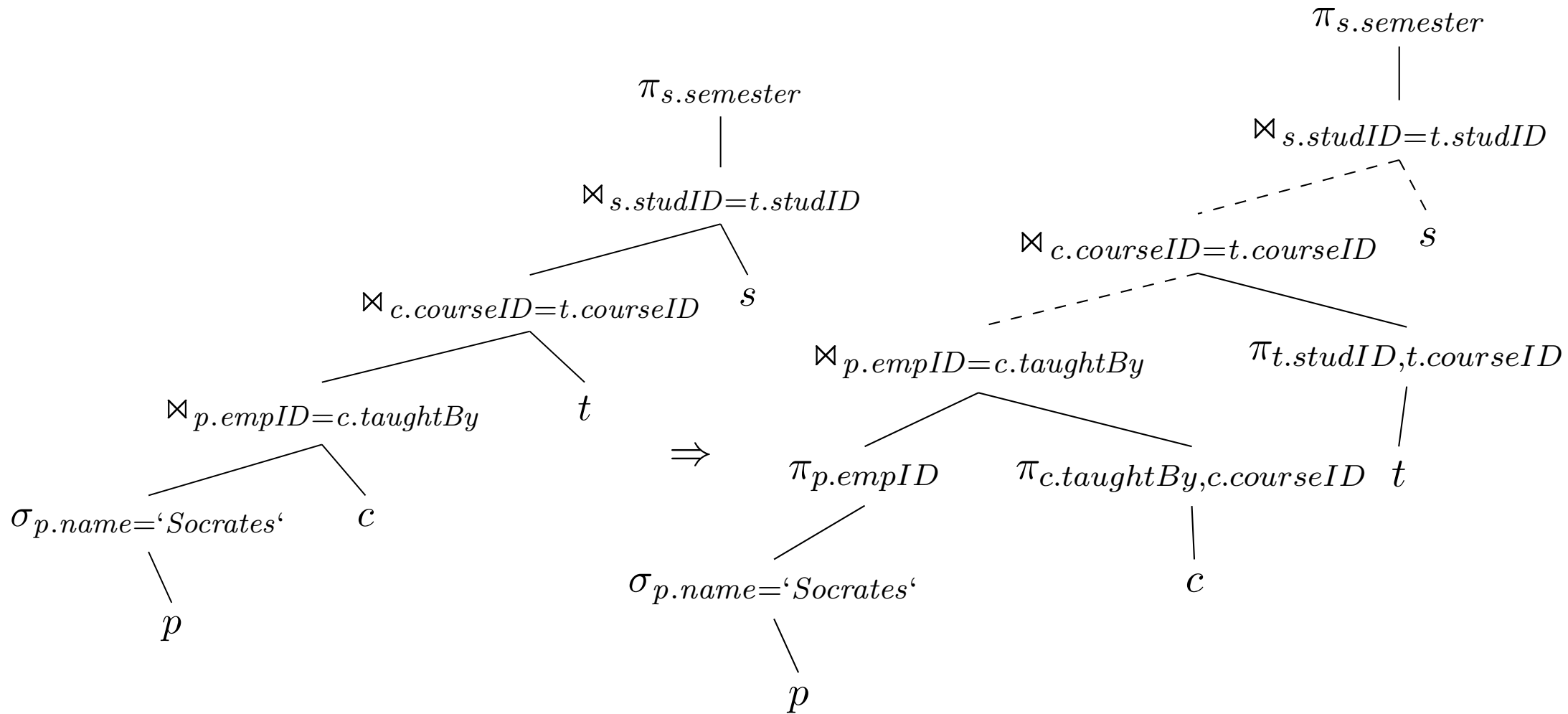


Effekt: Verkleinerung der Zwischenergebnisse



Schätzung der Zwischenergebnisgröße nur mit Statistiken möglich
→ Kostenmodelle

Einführen und verschieben von Projektionen



Vorsicht

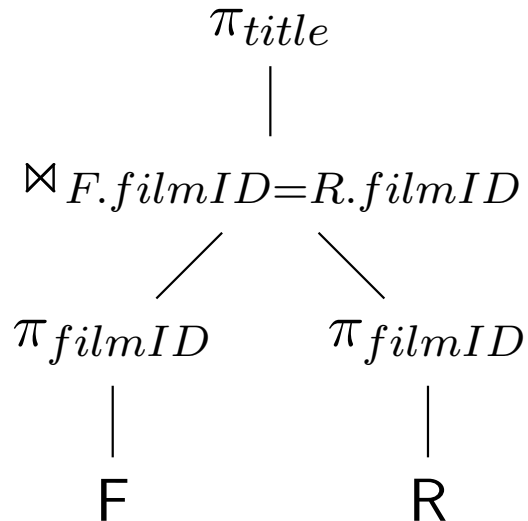
Finde die Titel von reservierten Filmen

```
SELECT DISTINCT title  
FROM film F, reserved R  
WHERE F.filmID = R.filmID
```

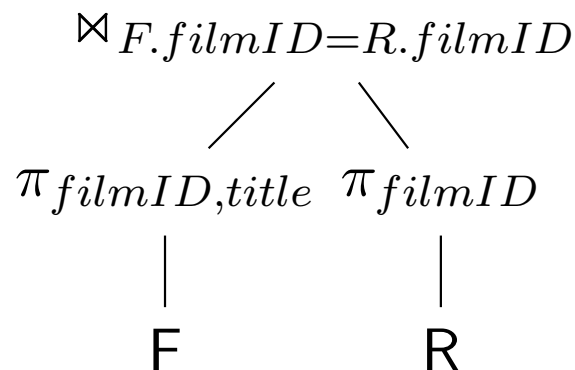
Vorsicht

Finde die Titel von reservierten Filmen

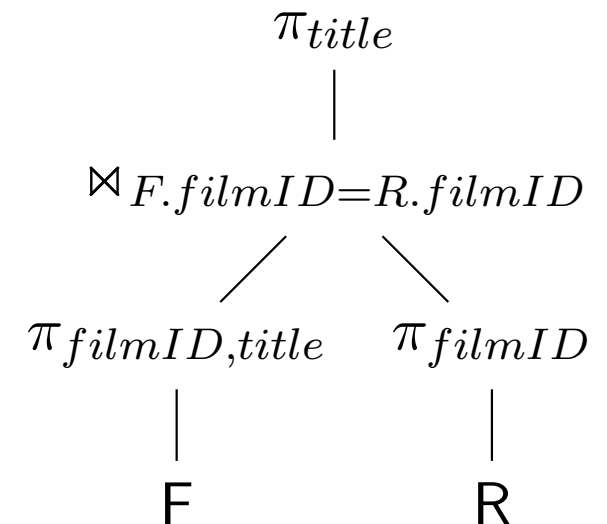
```
SELECT DISTINCT title
FROM film F, reserved R
WHERE F.filmID = R.filmID
```



Zu viel Projektion



Zu wenig Projektion

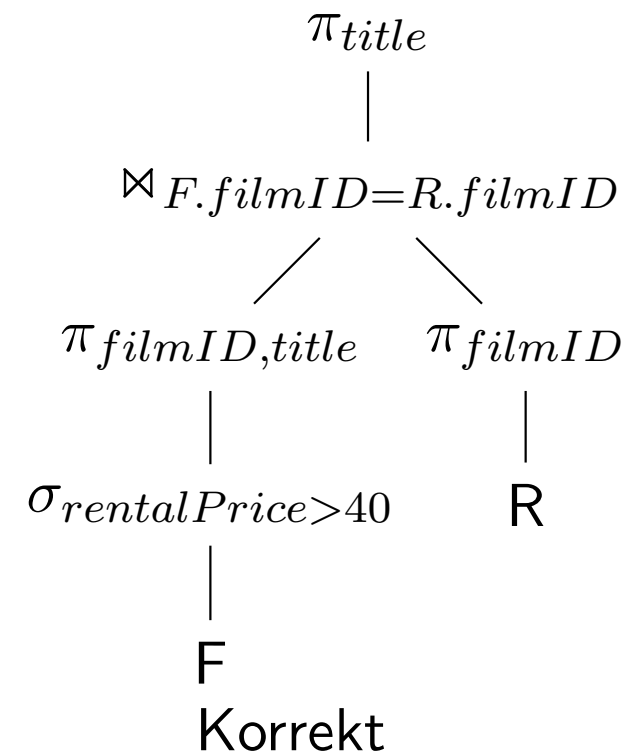
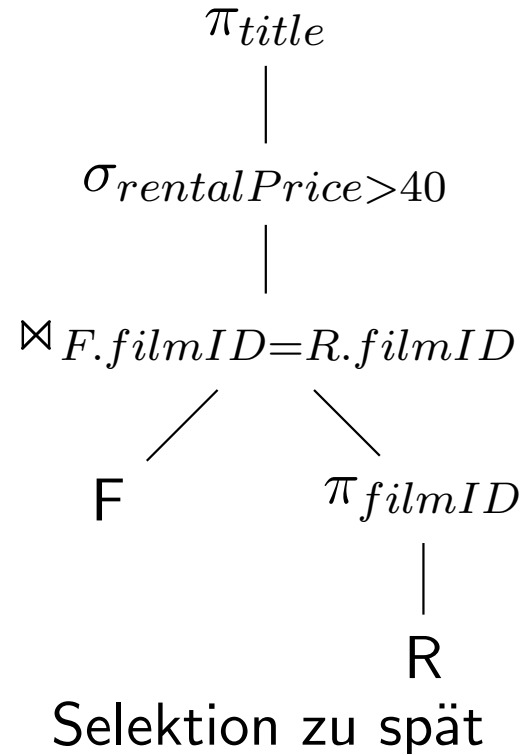
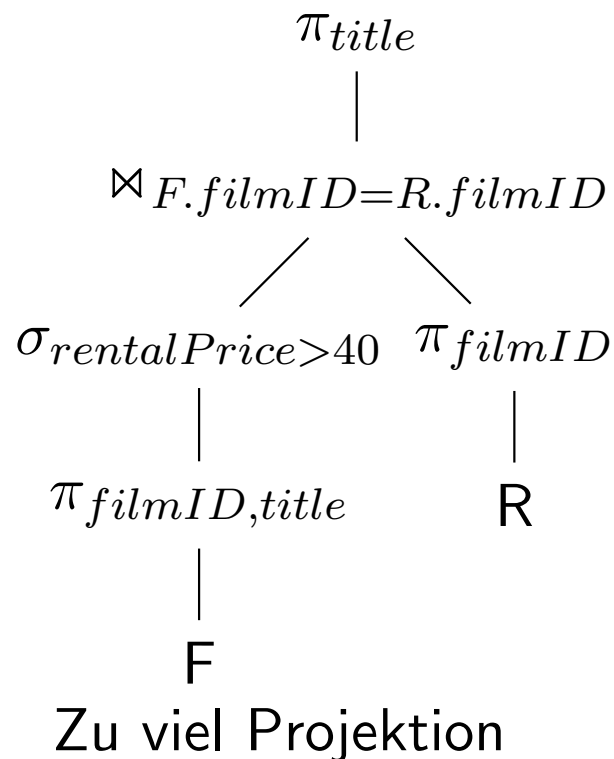


Korrekt

Vorsicht

Finde die Titel von teuren reservierten Filmen

```
SELECT DISTINCT title
FROM film F, reserved R
WHERE F.filmID = R.filmID AND F.rentalPrice > 40
```



- 3 Implementierung von Operatoren
 - Selektion (Access Paths)
 - Join-Algorithmen

Beispieldatenbank

- customer (customerID, name, street, city, state)
- reserved (customerID, filmID, resDate)
- film (filmID, title, kind, rentalPrice)

Select-Anfragen

- Primary Key, Punkt

$$\sigma_{filmID=2}(film)$$

- Punkt

$$\sigma_{title='Terminator'}(film)$$

- Bereich

$$\sigma_{1 < rentalPrice < 4}(film)$$

- Konjunktion (logisches und)

$$\sigma_{kind='F' \wedge rentalPrice=4}(film)$$

- Disjunktion (logisches oder)

$$\sigma_{rentalPrice < 2 \vee kind='D'}(film)$$

Selektion (Access Paths)

Selektion

Hauptziel

Ersetze die Blattoperatoren im Anfrageplan durch konkrete Zugriffsmethoden.

Selektion – Punkt-/Bereichsanfragen

Linear Search

- Aufwändig, funktioniert aber immer

Binary Search

- Nur wenn die Datei entsprechend sortiert ist

Primary Hash Index

- Single Record Retrieval – funktioniert nicht für Bereichsanfragen

Primary/clustering Index

- Mehrere Records für jeden Wert
- Pointer zum Block mit dem ersten Record mit gesuchtem Wert

Secondary Index

- Jeder Record hat eigenen Pointer
- Kann teuer werden

Strategien für konjunktive Anfragen

```
SELECT *  
FROM customer  
WHERE name = 'Jensen' AND street = 'Elm'  
      AND state = 'Arizona'
```

- Kann ein Index auf (name) oder (street) benutzt werden? Ja
- Kann ein Index auf (name, street, state) benutzt werden? Ja
- Kann ein Index auf (name, street) benutzt werden? Ja
- Kann ein Index auf (name, street, city) benutzt werden? Ja
- Kann ein Index auf (city, name, street) benutzt werden? Nein

Optimierung von konjunktiven Anfragen

Indexe bieten gute Möglichkeiten, die Performanz zu verbessern

Strategien für konjunktive Anfragen

- Existierende Indexe verwenden
 - Ideal: Es gibt einen passenden Index definiert auf allen benötigten Attributen
 - Falls es mehrere Indexe gibt
 - benutze den selektivsten Index, danach übrige Konditionen auswerten
- Überschneidung von Pointern ausnutzen (wenn mehrere Indexe anwendbar sind)
 - Index Lookups, um relevante Pointer zu erhalten
 - Überschneidung der Pointer ermitteln (Konjunktion der Bedingungen)
 - Die zugehörigen Records/Tupel lesen

- 3 Implementierung von Operatoren
 - Selektion (Access Paths)
 - Join-Algorithmen

Join-Algorithmen

Algorithmen

- Nested Loop Join
- Index-based Join
- Sort-Merge Join
- Hash Join

Strategien basieren auf Blöcken (nicht Tupel) als Basis

- Schätze I/Os (Block Retrievals)
- Benutze Puffer (Buffer) im Hauptspeicher

Tabellengröße und Join-Selektivität bestimmen die Kosten

- Selektivität einer Anfrage: $sel = \frac{\#tuples\ in\ result}{\#candidates}$
- Für einen Join entspricht $\#candidates$ der Größe des Kartesischen Produkts

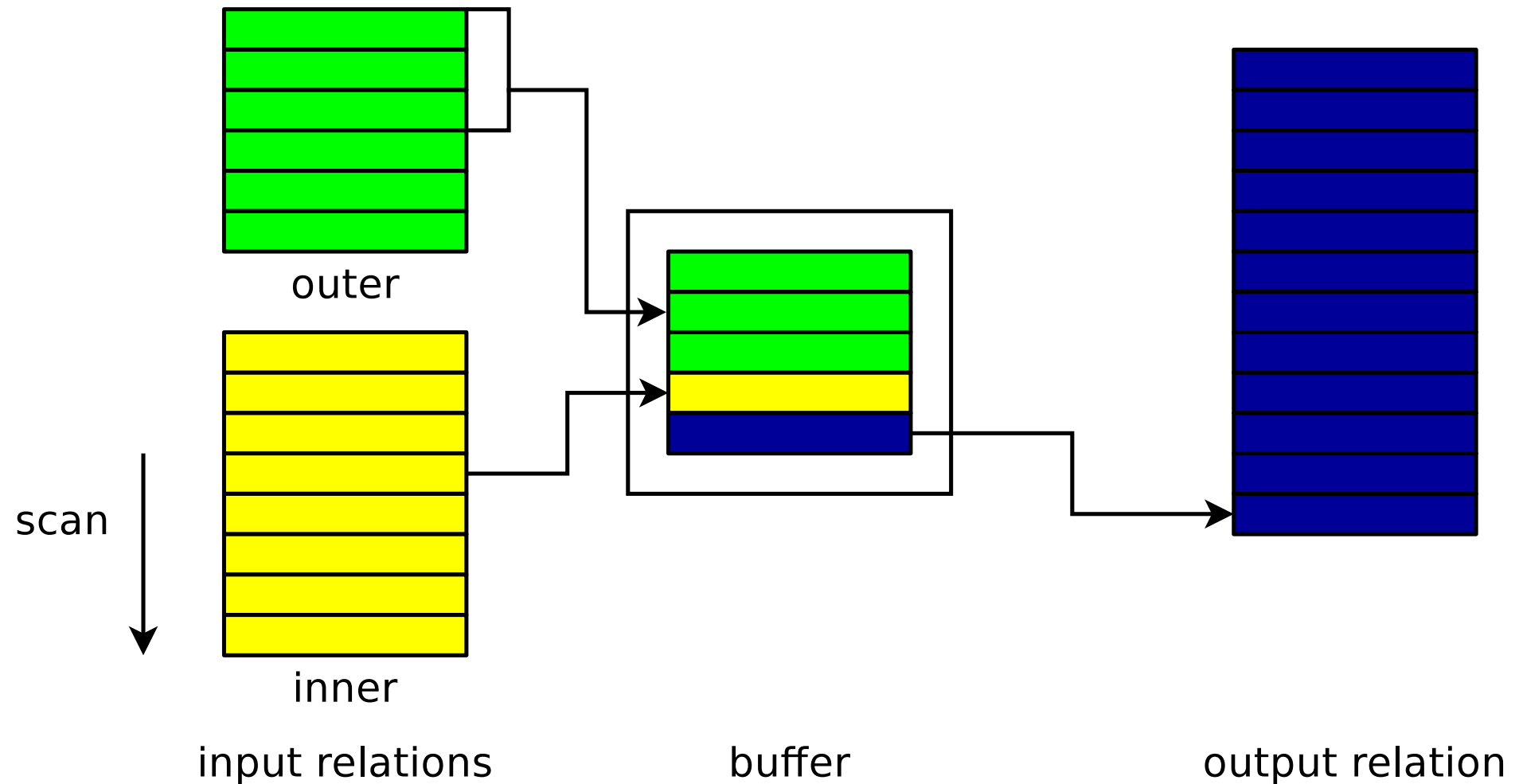
Join-Algorithmen

Nested Loop Join

ID	name		number	ID		ID	name	number
10	Jim		100	23		10	Jim	110
13	Joe		110	10		13	Joe	150
14	Sue		120	15		15	Pete	120
15	Pete	⋈	130	23	=	15	Pete	160
21	Dave		140	23		21	Dave	170
23	Anne		150	13		23	Anne	100
			160	15		23	Anne	130
			170	21		23	Anne	140
emp			phone			result		

- Brute-force Strategie, teurer, alle Tupel werden miteinander verglichen
- Kein Preprocessing der Input-Relationen
- Kein Index nötig, alle Joinbedingungen werden unterstützt

Block Nested Loop Join



Block Nested Loop Join

Nicht alle Blöcke beider Relationen passen gleichzeitig in den Hauptspeicher

repeat

 lese $n_B - 2$ Blöcke der äußeren Relation

repeat

 lese 1 Block der inneren Relation

 Vergleiche enthaltene Tupel

until innere Relation ist vollständig durchlaufen

until äußere Relation ist vollständig durchlaufen

Parameter

- b_{inner}, b_{outer} : Anzahl von Blöcken
- n_B : Größe des Buffers im Hauptspeicher

Kostenschätzung (Block Transfers)

$$b_{outer} + (\lceil b_{outer} / (n_B - 2) \rceil) \cdot b_{inner}$$

Wenn wir weitere Systemparameter (Block-Transfer, Disk Seek, CPU Speed, ...) und die Größen der Relationen kennen, können wir die Berechnungszeit abschätzen.

Block Nested Loop Join

Example (*reserved* ⋈ *customer*)

- Anzahl von Blöcken
 $b_{reserved} = 2.000, b_{customer} = 10$
- Größe des Buffers im Hauptspeicher
 $n_B = 6$
- Kostenschätzung (Block Transfers)
 $b_{outer} + (\lceil b_{outer} / (n_B - 2) \rceil) \cdot b_{inner}$

Kosten

- reserved als äußere Relation
 $2.000 + \lceil (2.000/4) \rceil \cdot 10 = 7.000$
- customer als äußere Relation
 $10 + \lceil (10/4) \rceil \cdot 2.000 = 6.010$